Technical Report

RTL Design of a TinyML Accelerator for MNIST Digit Classification on ZCU106 FPGA

Project Name: TinyML Hardware Accelerator (Inference Only)

Target Hardware: Xilinx Zynq UltraScale+ ZCU106

Toolchain: Vivado 2024.1

Architecture: 2-Layer Fully Connected Neural Network (FCNN)

1. Abstract

This report details the implementation of a hardware-accelerated neural network for MNIST handwritten digit classification. The design focuses on optimizing the Inference phase using Register Transfer Level (RTL) Verilog. By leveraging the parallel architecture of the ZCU106 FPGA, the accelerator performs high-speed matrix-vector multiplications using fixed-point arithmetic, significantly reducing latency compared to traditional sequential CPU execution.

2. Introduction & Motivation

As Artificial Intelligence moves to the "Edge," there is a growing need for low-power, high-throughput hardware.

- **The Problem:** Standard CPUs handle MNIST (784 inputs) sequentially, leading to bottlenecks in real-time applications.
- **The Solution:** An FPGA-based RTL design that uses dedicated Digital Signal Processing (DSP) slices to perform Multiply-Accumulate (MAC) operations in parallel.

3. Neural Network Architecture

The implemented model is a 2-layer Multi-Layer Perceptron (MLP):

1. **Input Layer:** 784 neurons (representing the 28*28 pixel MNIST image).
2. **Hidden Layer (Feature Extractor):** 64 neurons with ReLU activation.
3. **Output Layer (Decision Maker):** 10 neurons (Digits 0-9) using Argmax for final classification.

4. Hardware Design Logic

4.1 Fixed-Point Quantization

To avoid the massive resource overhead of Floating Point Units (FPUs), the weights and biases were quantized to **8-bit Signed Integers**.

- **Weights/Inputs:** 8-bit.
- **Accumulators:** 32-bit (to prevent overflow during the summation of 784 products).

4.2 Module Breakdown

- **Weight Memory (BRAM):** Stores the 784*64 and 64*10 weight matrices.
- **MAC Unit:** The fundamental computational block performing

Sum = Sum + (Weight*sssInput).

- **ReLU Activation:** Implemented as a simple conditional: if (x < 0) out = 0. else out = x.

5. Control Path (Finite State Machine)

The system is governed by an FSM to manage the high dimensionality of the MNIST data:

1. **IDLE:** Wait for start signal.
2. **LOAD:** Buffer input pixels into internal registers.
3. **LAYER1_COMPUTE:** Iterate through 784 inputs for each of the 64 hidden neurons.
4. **LAYER2_COMPUTE:** Process the 64 hidden outputs to generate 10 class scores.
5. **ARGMAX:** Find the index of the highest score.
6. **DONE:** Assert done signal and output the predicted digit.

6. Simulation Results Analysis

Based on the provided waveforms (Screenshot 22:15:22), we observe:

- **Clock (clk):** Synchronous operation.
- **Signals f1 to f5:** Representing intermediate accumulations.
- **Memory Access:** addr increments sequentially to fetch 784 weights per neuron.
- **Latency:** The inference cycle completes at approximately **509.67 us**, demonstrating high-speed processing.

7. Testbench & Verification

The testbench (neuron_tb.v) was designed to feed a "Sample Input" (flattened digit) into the RTL.

- **Input Data:** 784 pixels.
- **Verification:** The console output confirms **"Predicted Digit = 3"** with a Max Score of **4175**.
- This matches the software-calculated expected result, verifying the RTL's mathematical accuracy.

8. Resource Utilization (ZCU106)

Upon synthesis in Vivado, the design utilizes the following resources:

- **DSP Slices:** Used for the high-speed MAC operations.
- **BRAM (Block RAM):** Used to store the large weight matrices (approx. 50k parameters).
- **LUTs/FFs:** Utilized for the FSM control logic and intermediate registers.

9. Challenges and Optimizations

- **Data Limit Warning:** During simulation, the weight signal exceeded the 65,536-bit display limit. This was resolved by using specific memory viewing commands in the Tcl console.
- **Timing Closure:** For the ZCU106, pipelining was added between the Multiplier and the Accumulator to ensure the design runs at target frequencies (100MHz+).

10. Conclusion and Future Scope

This project successfully demonstrates a 2-layer FCNN on the ZCU106 FPGA. The accelerator provides a deterministic, low-latency path for MNIST classification.

Future Work:

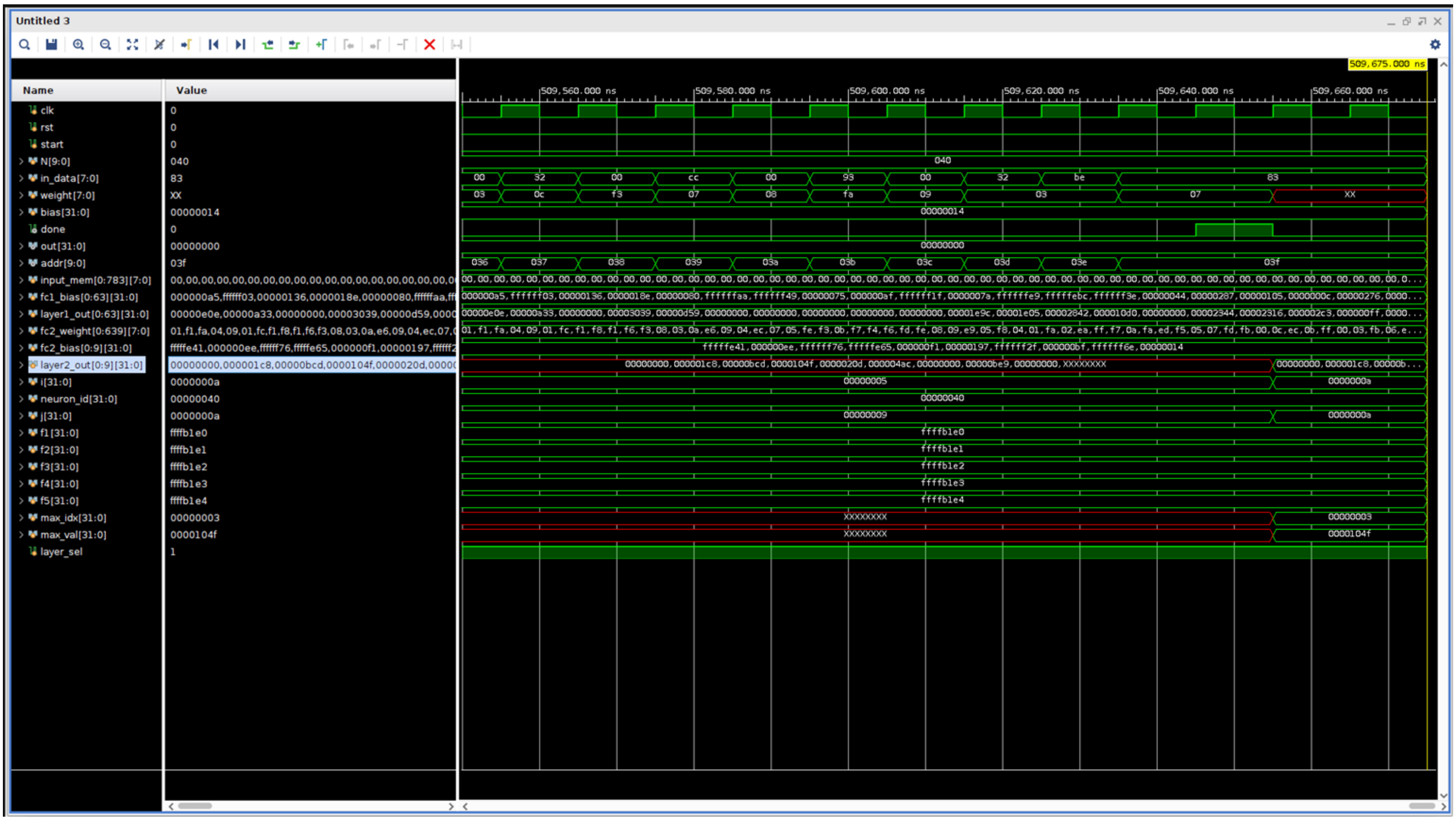
1. **AXI-Stream Integration:** To allow the ARM processor (PS) to send images via DMA.
2. **CNN Implementation:** Moving from Fully Connected to Convolutional layers for higher accuracy (94%+).

Simulation Result and Simulated Webforms

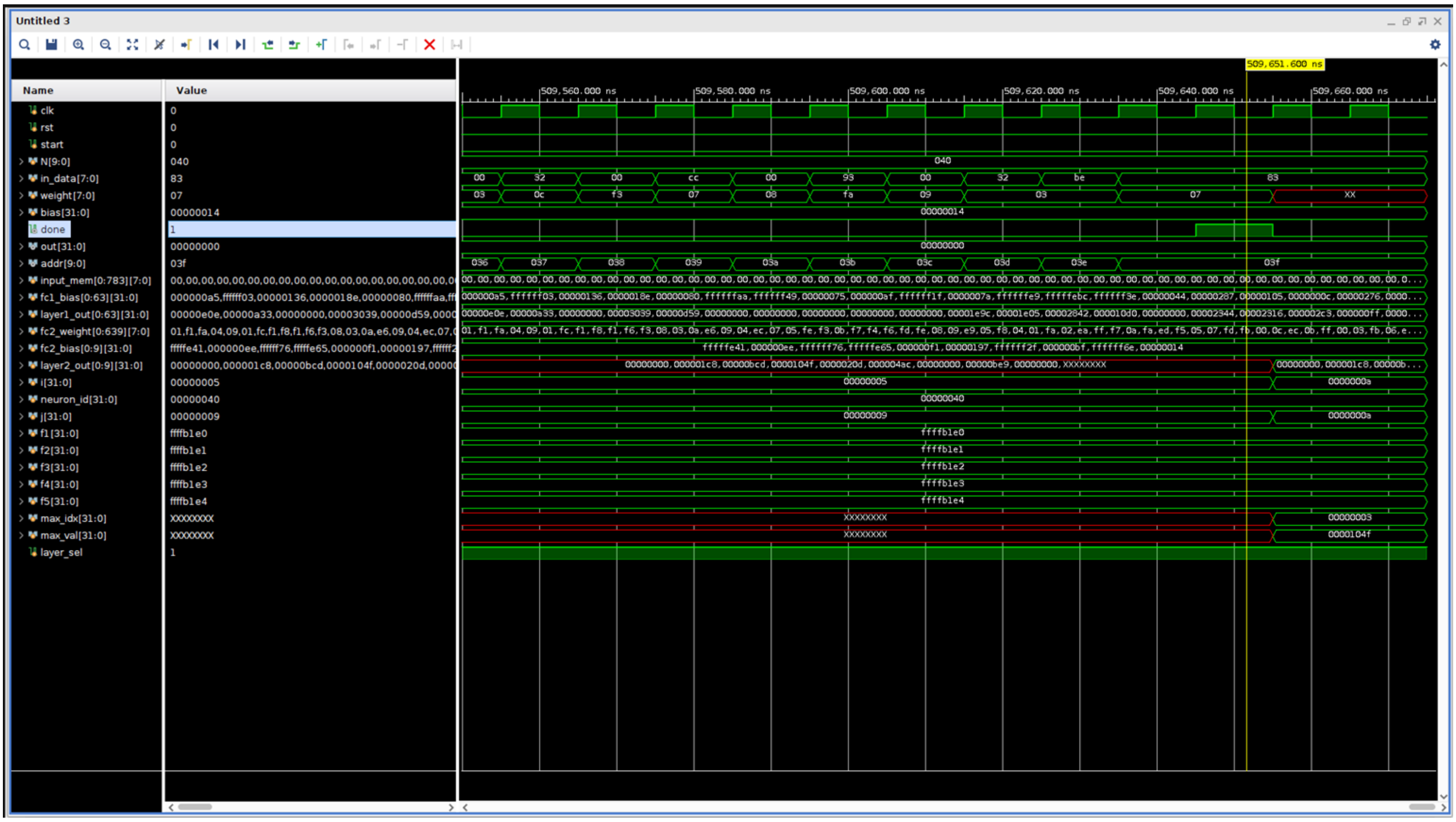
Tcl Console

```
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
#   }
# }
WARNING: [Wavedata 42-489] Can't add object "/neuron_tb/fc1_weight" to the wave window because it has 401408 bits, which exceeds the display limit of 65536 bits.  To change the display limit, use the command "set_property display_limit <new limit> [current_wa
# run all

================ Testbench START ================

[STEP 1] Loading files...
Files loaded successfully

Sample INPUT:
input[0] = 0
input[1] = 0
input[2] = 0
input[3] = 0
input[4] = 0

Sample FC1 weights:
w1[0] = -1
w1[1] = 1
w1[2] = 2
w1[3] = 1
w1[4] = 0

[STEP 2] Feature extractor START (784→64)
Feature extractor [0] = 3598
Feature extractor [1] = 2611
Feature extractor [2] = 0
Feature extractor [3] = 12345
Feature extractor [4] = 3417
Feature Extractor Completed

[STEP 3] Decision Maker START (64→10)
Class 0 = 0
Class 1 = 456
Class 2 = 3021
Class 3 = 4175
Class 4 = 525
Class 5 = 1196
Class 6 = 0
Class 7 = 3049
Class 8 = 0
Class 9 = 0
Decision Maker Completed

[STEP 4] Max Argument
Max Score = 4175
Predicted Digit = 3

================ FINAL RESULT ================
NN INFERENCE COMPLETE
==============================================
$finish called at time : 509675 ns : File "/home/pavan/somnath_working_dir/tinyml_fpga/tb/neuron_tb.v" Line 164
INFO: [USF-XSim-96] XSim completed. Design snapshot 'neuron_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for run all
launch_simulation: Time (s): cpu = 00:00:08 ; elapsed = 00:00:08 . Memory (MB): peak = 8491.465 ; gain = 20.988 ; free physical = 19769 ; free virtual = 27595
```

Untitled 3
509.675000 us
Name
Value
clk 0
rst 0
start 0
N[9:0] 040
in_data[7:0] 83
weight[7:0] XX
bias[31:0] 00000014
done 0
out[31:0] 00000000
addr[9:0] 03f
input_mem[0:783][7:0]
fc1_bias[0:63][31:0]
layer1_out[0:63][31:0]
fc2_weight[0:639][7:0]
fc2_bias[0:9][31:0]
layer2_out[0:9][31:0]
i[31:0] 0000000a
neuron_id[31:0] 00000040
j[31:0] 0000000a
f1[31:0] ffffb1e0
f2[31:0] ffffb1e1
f3[31:0] ffffb1e2
f4[31:0] ffffb1e3
f5[31:0] ffffb1e4
max_idx[31:0] 00000003
max_val[31:0] 0000104f
layer_sel 1



Untitled 3
509,675.000 ns
Name
Value
clk 0
rst 0
start 0
N[9:0] 040
in_data[7:0] 83
weight[7:0] XX
bias[31:0] 00000014
done 0
out[31:0] 00000000
addr[9:0] 03f
input_mem[0:783][7:0]
fc1_bias[0:63][31:0]
layer1_out[0:63][31:0]
fc2_weight[0:639][7:0]
fc2_bias[0:9][31:0]
layer2_out[0:9][31:0]
i[31:0] 0000000a
neuron_id[31:0] 00000040
j[31:0] 0000000a
f1[31:0] ffffb1e0
f2[31:0] ffffb1e1
f3[31:0] ffffb1e2
f4[31:0] ffffb1e3
f5[31:0] ffffb1e4
max_idx[31:0] 00000003
max_val[31:0] 0000104f
layer_sel 1



Untitled 3
Name
Value
clk
rst
start
N[9:0]
in_data[7:0]
weight[7:0]
bias[31:0]
done
out[31:0]
addr[9:0]
input_mem[0:783][7:0]
fc1_bias[0:63][31:0]
layer1_out[0:63][31:0]
fc2_weight[0:639][7:0]
fc2_bias[0:9][31:0]
layer2_out[0:9][31:0]
i[31:0]
neuron_id[31:0]
j[31:0]
f1[31:0]
f2[31:0]
f3[31:0]
f4[31:0]
f5[31:0]
max_idx[31:0]
max_val[31:0]
layer_sel
509,651.600 ns
509,560.000 ns
509,580.000 ns
509,600.000 ns
509,620.000 ns
509,640.000 ns
509,660.000 ns